

An instance for the `Car` class can be created using the `new()` function. For example:

```
> m <- new("Car",
+   mpg=21.4,
+   cyl=6,
+   disp=258)

> m
An object of class "Car"
Slot "mpg":
[1] 21.4

Slot "cyl":
[1] 6

Slot "disp":
[1] 258
```

The `setValidity()` function is used to add validity checks for the members in the class. For example, we can check that the number of cylinders for the cars should be less than eight, as shown below:

```
> setValidity("Car",
+   function(obj) {
+     obj@cyl <= 8
+   }
+ )

Slots:

Name:      mpg      cyl      disp

Class: numeric numeric numeric
```

The `validObject()` function can then be applied to check if any object conforms to the class specification, as shown below:

```
> validObject(m)
[1] TRUE
```

If we try to create a `Car` object with ten cylinders, the command fails. For example:

```
> invalid <- new("Car",
+   mpg=18.7,
+   cyl=10,
+   disp=360)

Error in validObject(.Object) : invalid class "Car"
object: FALSE
```

The object members can be obtained directly or by using a function, as illustrated below:

```
> m@mpg
[1] 21.4

> getmpg <- function(object) {object@mpg}
> getmpg(m)
[1] 21.4
```

You can also use generic functions in R that support polymorphism using the `setGeneric()` function. This function accepts the following arguments:

Argument	Description
name	A name for the generic function
def	An optional definition for the function
group	An optional group generic name to be associated with the function
valueClass	An optional name of the class to which the function should belong
where	The environment in which to store the generic function
package	A package name
signature	Names of formal arguments and classes for the same
simpleInheritanceOnly	A logical value to use simple inheritance only
useAsDefault	A logical value to indicate if to use as the default method

The `showMethods()` function displays the function, environment and its associated objects, as indicated below:

```
> showMethods(disp)
Function: disp (package .GlobalEnv)
object="ANY"
object="Car"
(inherited from: object="ANY")
```

You can also define your own methods using the `setMethod()` function. In the following example, a `show()` method is added to the `Car` class.

```
> setMethod("show",
+   "Car",
+   function(obj) {
+     cat("mpg: ", obj@mpg, "\n")
+   })
```